

Transcript

Module 15: Introduction to Deep Learning

Video 1: Introduction to Deep Learning

Introduction to Deep Learning. Welcome to this overview of deep learning, a powerful subset of machine learning as depicted in our introductory slide. Deep learning, integral to the broader landscape of AI specialises in interpreting and processing complex data structures leading, innovations in the field such as computer vision and natural language processing.

This week we will delve into the essence of deep learning, exploring its applications in computer vision through convolutional neural networks. We will also discuss how CNN's function and demonstrate their capabilities with a Keras example, which is focused on object classification. This practical illustration shows how CNN interpret visual data to identify and categorise objects. Shifting from image to text, we also examine deep learning's role in processing languages. Here, recurrent neural networks (RNN) come into play, which is designed to handle sequential information making them ideal for tasks like text classification. Keras tutorial will help illustrate how RNNs can predict and generate text, offering insights into their operational mechanisms.

By connecting these concepts back to our initial diagram, it's clear that deep learning forms a critical component of machine learning, pushing the boundaries of what AI can achieve in various domains.

Video 2: What is Deep Learning?

What is deep learning? Welcome to our exploration of deep learning, a transformative branch of machine learning that's reshaping how technology interprets the world around us. Today we will dive into what deep learning is, its advantages over traditional machine learning and why it's become so pivotal in very recent years.

What is deep learning? Deep learning is a specialised branch of machine learning. It functions through layers that processes data, applying complex mathematical transformations. These layers structured in stacks, learn representations directly from data without needing explicit programming for feature detection. Each layer's parameters known as weights are adjusted during the training to minimise a loss function, redefining the model's accuracy. This is similar to adjusting the weights in neural network, depending on the error between Y and $\hat{Y}$ value. By leveraging such layers, deep learning can perform task ranging from image recognition to language processing making it a powerful tool in modern AI solution.

Let's visualise this with an example. Imagine a simple, deep neural network designed for digit recognition. Our input is an image of handwritten digit. This image passes through multiple layers of the network, each refining and focusing on essential features like edges, curves, until the final layer outputs a prediction. On the left-hand side, you see an image of number four, which serves as an input. This image is processed through multiple hidden layers before reaching the output layer on the right. In the output layer, the node corresponding to the number four will produce a significantly high value, indicating that the input is recognised as a number four. Now the question arises. What is happening within these layers in the network?

Think of deep learning as a multi-stage information distillation process where data passes through successive filters becoming increasingly refined and useful.

The most pertinent information is transmitted to the output layer for classification. The transformations and feature extraction conducted by each layer are visible in this illustration.

In deep learning, this represents a multi-stage approach to learning data representation that is both straightforward and scalable. This process akin to distillation extracts, and amplifies useful information while diminishing irrelevant data, enabling precise classification.

You might be wondering why we use deep learning instead of traditional machine learning for certain tasks. Let's try to understand the concept of feature extraction to clarify this. Feature extraction is a complex challenge, particularly in object recognition tasks like face detection. In traditional machine learning approaches, you must manually define and input specific features such as eyes, ears, and nose. The machine learning algorithm then determines which features are more significant for distinguishing between different individuals. In contrast, while using deep learning for face detection, the deep learning framework automatically identifies the important features necessary for recognising faces, provided it has access to a large amount of data. This automatic feature extraction capability for deep learning reduces the need for manual feature specification and makes it highly effective for complex recognition tasks.

Why is deep learning popular now? You might be wondering why deep learning has surged its popularity only recent years rather than 20 years ago when many of the frameworks we use today have been initially designed. Let me explain the factors contributing to its rise. The first one is the advancement in computing infrastructure. Though the theoretical framework for deep learning were developed decades ago, their practical applications was limited due to the lack of powerful computing infrastructures. In recent years, advancements in technologies like NVIDIA's, GPUs called as graphical processing units for parallel computing. Google's TPU called tensor processing units and cloud computing platforms like AWS and Google Colab have revolutionised this field. These technologies allow for parallel processing of complex tasks. Such as performing hundreds of dot products simultaneously with GPUs making real-time deep learning feasible. The second factor is the availability of large datasets: The explosion of data from smartphone and IoT devices as provided the vast amount of data required for training deep learning models effectively. Deep learning excels in environments where data is abundant using this data to improve and refine the model accuracy continuously.

The third factor contributing is open- source tools and pre-trained models. The accessibility of open-source frameworks such as Keras, PyTorch, and TensorFlow, along with the availability of pre-trained models, has democratised this deep learning. These resources allow even small companies to leverage advanced deep learning models without the need of extensive data or resource investment. Pre-trained models are deep learning models trained on large datasets that can be fine-tuned or adapted to specific tasks, significantly reducing the development time and computational resources. The next one is the superior performance with large datasets. Unlike traditional machine learning algorithms that stagnates. In performance after a certain amount of data, the deep learning model generally continue to improve as more data is provided. This graph illustrates that the amount of training data increases. The performance of the deep learning model steadily improves, whereas other algorithms tend to saturate.

These factors combined have propelled deep learning to the forefront of AI research and application, making it a fundamental tool in tackling complex computational problems that were previously considered infeasible.

In conclusion, deep learning represents a significant leap forward in our ability to harness the power of data for more intelligent decision-making by automating the feature extraction and driving on large datasets. Deep learning is not just an academic curiosity, but a powerful tool driving progress in the fields from autonomous driving to medical diagnostics.

Video 3: Applications of Deep Learning for Computer Vision

What are the applications of deep learning in computer vision? In this video, we explore several compelling applications of deep learning in the field of computer vision. These advanced technologies enable computers to interpret and understand visual world, much like humans do, but with greater speed and accuracy.

Let's begin with object detection. Deep learning models excel at identifying and locating various objects within an image. This capability is crucial for systems ranging from surveillance to advanced driver assistance. In a practical demonstration, you can see how the model accurately detects different objects, highlighting each with distinct bounding boxes. Next is the image segmentation. This technology allows for detailed differentiation and segmentation of objects within an image. For instance, in a dynamic scene, a deep learning model can distinguish and segment a person and the umbrella they are carrying, assigning different colours to each segment. This functionality is vital for applications such as autonomous driving where the vehicle must understand and react to its environment comprehensively. Another exciting application is style transfer. This process involves taking artistic style of one image and applying it to another, transforming the appearance while retaining the original content. This technique is already popular in many smartphone apps which apply artistic filters to your photos.

We look at the automatic image colouring technique. Imagine transforming historical black and white photos into vibrant colour images. Deep learning can analyse these photographs and intelligently add colours that are realistic and historically plausible. Image inpainting is another innovative use where the model fills in missing or damaged parts of images. Whether it's restoring old photos or repairing areas with missing pixels, deep learning models can create the lost content with surprising accuracy. Image super-resolution. Image super-resolution is a technique that enhances the resolution of an image. For example, a deep learning model can take an 8x8 pixel image and upscale it to a 32x32 pixel resolution without losing clarity. This is particularly useful for enhancing old films or low-quality videos to a higher resolution.

Image Synthesis. Image synthesis represents a leap in generating new content. Here, deep learning can transform an image of a zebra into a horse, or vice versa, showcasing its ability to modify and synthesise entirely new visual content from existing images. Lastly, the generative models, which are capable of creating detailed and diverse images of objects like bedrooms from scratch. These images are not captured by cameras. But are synthesised by deep learning algorithms, demonstrating the model's ability to understand and generate complexity.

These applications are just the tip of the iceberg, showing how deep learning continues to push the boundaries of what's possible in computer vision, transforming the industry's and enhancing our interaction with technology.

Video 4: Deep Learning for Computer Vision

Deep learning for computer vision. Welcome to our exploration of how deep learning revolutionise computer vision. In this video, we will uncover why convolutional neural networks, often called as CNNs are foundational to advancing this dynamic field.

Computer vision is the cornerstone of deep learning success. From Google's image search to self-driving cars and automated classification systems, CNN's are integral to these technologies. Unlike traditional artificial neural network that learns global features, CNN excels in picking up local nuance features. This capability makes them exceptionally efficient for visual tasks.

Let's delve into what makes CNNs unique. The first thing to note is that CNN's learn translational invariant patterns. This means once a CNN learns to recognise a feature like a diagonal line at one corner of an image, it can identify the same line anywhere else in the picture. This ability is pivotal because it allows CNNs to recognise objects in various positions and orientations enhancing their versatility across different scenarios. Another significant advantage is their ability to learn spatial hierarchies of patterns. Imagine the process of recognising a cat. In the initial layers, a CNN might detect simple edges and lines. The subsequent layers integrate these initial findings to discern more complex patterns like parts of a face. As we progress deeper, these features merge into higher level representations like nose, eyes, eventually recognising the entire cat. This hierarchical learning mirrors how our own visual cognition identifies and processes the scenes.

Now, you might wonder why not use densely connected neural network for images? The answer lies in the efficiency and specificity. Densely connected neural networks treat input data uniformly without regard to spatial hierarchies. Leading to less efficient learning process for images where the arrangement and the relationship of pixels are crucial. The structure in CNN allows them to handle the vast array of data encountered in real world scenarios, from crowded street scenes in autonomous driving to intricate variations in video frames for security surveillance. This efficiency is due to their ability to learn and re-apply features anywhere in the image, dramatically reducing the computational load and making them faster and more scalable.

As we wrap up, remember that the magic of CNN lies in their ability to take simple concepts like lines and edges and build up on complex understanding, much like constructing a building from bricks. In our next video, we'll break down these processes further showcasing how CNN's not only learn but excel at interpreting our visual world.

Video 5: What is Convolutional Neural Network (CNN)?

What is CNN? Let's begin by understanding what a convolutional neural network is. Today we are going to look at AlexNet, a model that revolutionised the way we approach machine learning tasks involving images. AlexNet was one of the first deep neural network that significantly outperformed earlier models on image classification tasks. It consists of multiple layers that each play a part in understanding the content of images.

The journey through AlexNet begins with a input layer here, an image of size, 64 x 64 pixels with three colour channels (red, green, and blue) is introduced into the network. Each pixel value in these channels' ranges from 0 to 255, providing the initial data that the network will process.

The core of AlexNet consists of several convolutional layers. The first layer applies 64 filters of size 5x5, which helps in detecting basic patterns such as edges and textures from the input image. Subsequent layers use smaller filters but increase in number enhancing the network's ability to learn more complex features. Each blue box here represents a deep convolutional layer where multiple filters are applied to extract various features. The size of filters typically starts from 5x5 and is reduced to 3x3 in deeper layers. Increasing the abstraction level of the information extracted. In these layers, filters are applied across the entire image, creating what we call feature maps. For example, the first convolutional layer might use 64 different filters each creating a separate feature map. This process is crucial for pattern recognition within the network.

Following the convolutional layer, AlexNet employs three fully connected layers. These dense layers combine all the features learned by the convolutional layer to classify the image into categories. The first two dense layer each has 4,096 nodes, which help in forming the high-level reasoning needed for classification. The final layer is a soft max layer, which outputs the probabilities of the image belonging to one of the predefined categories.

This layer makes the final decision in the classification process based on the complex features extracted and combined in previous layers. To summarise, AlexNet takes an input image and processes it through multiple convolutional and fully connected layers. Each designed to extract and condense information leading up to a classification output. As a data scientist, configuring the number of layers, the size of filters, and the number of nodes in each layer are critical factors that determine the network's performance. Understanding and setting up a network like AlexNet requires not just knowledge on neural network architectures but also experience in tuning parameters for optimal performance.

Let's try to understand more about convolutional operation that is done in convolutional neural network. "At the heart of convolution layer, we find what's known as convolutional kernels or simply filters. These are small matrices of real numbers that play a crucial role in processing the input image". "Imagine a filter as a 3x3 matrix. This is not just any matrix. It's designed to detect specific features in the image, such as edges, colours or textures. Here's how it works". The filter slides are called as strides across the input image. At each position, it performs a mathematical operation called the dot product between the filter values and the pixel values it covers. This operation accumulates into a single value that forms a part of the output image, highlighting the features the filter is designed to capture. This process isn't random; each filter is tailored to capture a specific part of the image's information. As the filter slides across the image, it systematically applies the same operation, ensuring that every piece of the input image is analysed. What's intriguing here is the role of padding. Padding is used to preserve the spatial dimension of the image. For example, the same padding adds zeros around the edges of the image, allowing the filters to operate fully even on the border pixels. The resulting output from this convolution process is what we call a feature map. This feature map is then passed on to the next layer in the network to undergo further processing. The entire operation is crucial for the CNN's ability to learn complex patterns in the data, making it powerful for tasks like image recognition.

Remember, the real power of convolution lies in its simplicity and efficiency, allowing networks like AlexNet to process visual information at multiple layers of abstraction. Let's see the animation of how Convolution works in a convolutional neural network (CNN). In this visualisation, the left matrix represents an input feature map possibly with different channels such as RGB. The central smaller matrix is a convolutional kernel or filter designed to detect specific features like edges or textures in the input. As this filter strikes over the input map, it performs element wise multiplication with the part of the input it covers, summing these products to yield a single output value in the resulting feature map, which is shown on the right. This process, essential for feature extraction in CNNs helps in tasks like image recognition.

This slide illustrates the process of defining a convolutional layer using Keras library. Here's a detailed breakdown of how this convolutional 2D layer is set up. This is the function call from Keras to create a convolutional layer. The parameter `filter=32`, indicates that this layer will have 32 filters (or kernels). Each filter extracts different kinds of features from the input image such as edges, textures, or colours. `kernel_size: kernel_size=3`, specifies that each filter is 3x3 in size. This size is effective for capturing the local patterns within the input, like small edges or corners.

Strides: stride is =1 means that the filter will move one pixel at a time over the input image, both horizontally and vertically. This stride setting allows the filter to comprehensively process every part of the image without skipping any pixels. Padding: Padding equal to same ensures that the output image size is the same as the input image size. It achieves this by padding the input images border with zeros, allowing the filter to operate at the edges of the input image.

Activation function. Activation equal to ReLU applies the rectified linear unit function to the output of the convolution operation. ReLU introduced non-linearity to the model by turning all the negative values into zero, which helps the network learn complex patterns effectively.

The filter parameters mentioned on the slide refers to the weights within the convolutional filters of Con2D layer in the Keras. These parameters are essentially trainable variables, meaning they are adjusted through the training process. The network learns the optimal values for these parameters based on the training data. This learning process enables the network to identify which features, like edges, textures or patterns are most important for performing tasks such as image recognition or classification. Essentially, these filter parameters help the network become better at detecting useful features in the input data, optimising its performance on the task at hand.

Pooling layer: In the next part of our discussion on CNN, we will explore the role of pooling layer essential component that contributes to the network's ability to understand and process images effectively. The pooling layer serves several crucial functions. Number one, reduction of computational load. By down sampling the feature maps pooling layer significantly reduce the number of parameters and computations in the network leading to faster processing. Number two, no trainable weights. Unlike convolutional layers, pooling layers do not have weights that learn during training. Instead, they use a deterministic process based on a sliding window mechanism to reduce spatial dimensions. Number three, use of aggregate functions. Pooling layers typically employ max or average pooling strategies. In max, pooling the maximum value within the certain window that is the kernel is selected and forwarded to the next layer, effectively capturing the most significant features. Average pooling, on the other hand, takes the average of the elements within that window.

Let's consider an example to illustrate how max pooling works with a 2x2 kernel and a stride of two without padding. This setup means that the kernel will move two pixels over for each operation covering non-overlapping regions of the input. This is done both horizontally and vertically. In the specific example, imagine a section of input feature map with values, [1, 5; 3,2]. The max pooling operation will scan this 2x2 area and selects the maximum value, which is five. Because we are using a stride of two, the kernel jumps to the next non-overlapping 2x2 area after each operation. This approach effectively reduce the spatial dimension of the resulting feature map by half, both in height and width. Assuming there is no padding to alter the dimensions. Consequently, the output from this operation would be similar feature map where each element is the maximum value of its corresponding 2x2 area in the input. Such a mechanism enhances the network's ability to detect features robustly regardless of small shifts and changes in the input image and also reduces the sensitivity to the exact location of features in the input space.

As we wrap up our exploration of CNN architectures, let's delve into the VGGNet, particularly the VGG 16 model from 2014, which represents a significant advancement in deep learning for computer vision. The architecture is straightforward: it starts with an input layer and takes an image. This followed by two convolutional layer with a small number of filters relative to more complex models, which are then followed by pooling layer. This pattern is repeated multiple times, increasing the number of filters each time before applying max pooling to reduce the spatial dimensions.

After several convolutional and pooling layers, the network transitions to three fully connected layers. The first two have 4,096 channels each while the third performs the classification into 1000 classes from ImageNet.

This is done by using a soft max activation function. This configuration makes VGGNet very adept at learning rich feature representation for a wide range of images.

The strength of VGGNet lies in its architecture leading to feature rich, deep convolutional network that is both versatile and widely applicable for various image recognition task. Despite its simplicity or perhaps because of it VGGNet reminds one of the most used and influential image recognition architecture in this field. This architecture, like many others, can be foundation for numerous project that requires image understanding and forms a critical part of the history and development of CNNs in deep learning.

To learn more about how VGGNet and other architectures are constructed and function. You might want to explore further detailed resources and implement these models in your own projects.

Video 6: Keras Example for Object Classification

In this video, we will explore how to construct a Convolutional Neural Network (CNN) using TensorFlow and Keras for object classification, specifically utilising this CIFAR-10 dataset. This dataset contains 60,000 32x32 colour images across 10 classes, making it ideal for developing and testing deep learning models in real-world scenarios.

Step one; imports and setups our journey begins with necessary imports. TensorFlow will serve as our framework for constructing deep learning models, while Keras provides a simpler interface for building and training these models. We also import Matplotlib for visualisation purposes and tools to help illustrate our model architecture. We then define several parameters that will dictate the training process. First, `batch_size`: this determines how many samples the network will see before making a weight update. `Epochs`: this is the number of times the entire dataset is passed forward and backward through the network.

Step two, loading and preparing data. The CIFAR-10 data is loaded directly from Keras's dataset repository, which conveniently splits it into training and testing sets. It's crucial to normalise the pixel values to 0 to 1 range to ensure consistent model training. Following data loading, we visualise some training images to understand the dataset's composition, which includes various classes like aeroplanes, cars, trucks, birds, and animals.

Step three, model creation. The architecture of our CNN involves several layers. Convolutional layer: These layers extract features from input images, using filters that slide across the image, spatially capturing patterns such as edges and textures. Conv2D layer parameters: First parameter is set to 32. This number specifies the depth of the output filter. (i.e, the number of output filters in the convolution). Second parameter is (3x3), which indicates the height and width of the 2D convolution window. (i.e the kernel size). The third parameter is the activation function. This specifies the activation function to use, which in this case is the rectified linear unit (ReLU). It introduces non-linearity to the decision process, allowing the model to learn more complex patterns. Pooling layers: these reduce the spatial dimensions of the output from convolution layers, which decreases the computation required and helps extract dominant features. Max pooling 2D layer parameters (2,2): This denotes the size of the polling window, reducing the spatial dimensions that is the height and width by the factor of two.

This operation simplifies the information by retaining only the maximum value in each window, which helps to reduce the computational load and model over fitting by extracting dominant features while discarding redundant information. Dense layers: a fully connected layer that interprets the features extracted by the convolutional and pooling layers to classify the image into one of the classes. The first dense layer has 64 neurons with ReLU activation function, whereas the second dense layer output layer has 10 neurons with soft max activation function. The soft max is appropriate as we are doing multi-class classification.

Step four, compile and train the model. The model is compiled using Adam Optimiser, a popular choice for deep learning applications. The loss function used is `sparse_categorical_crossentropy`, which is appropriate for multi-class classification problems where each class is mutually exclusive. The `model.fit()` function is used to train network on the training data. It takes the training data input, which is (X train) and labels, which is (Y train), and repeatedly adjust the model weights to minimise the loss function over number of epochs specified. The `batch_size` parameter controls the number of samples per gradient update while validation data is used to evaluate the model's performance on a separate dataset after each epoch, allowing us to monitor how well the model is generalising to new data outside its training set.

Step five, evaluation and visualisation. After training, we evaluate the model on the test set and visualise the training process to understand the accuracy and loss over the epochs, which helps to identify if the model is overfitting or underfitting. The plots show the training and validation, accuracy and loss over five epochs for a CNN model. In the model accuracy plot, both training and validation accuracy steadily increase over epoch, indicating that the model is learning and generalising well. In the model loss plot, both training and validation loss decreases, which also supports that the model is progressively learning from the training data and performing better on the validation data as well.

This comprehensive guide showcases how to build, train, and evaluate a CNN model for object classification using CIFAR-10 dataset. This demo also equips viewers with the knowledge to apply these techniques to other visual data challenges as well.

Video 7: Applications of Deep Learning for NLP

In this video, we will discuss the transformative impact of deep learning on natural language processing. As we interact daily with digital platforms, it is fascinating to see how deep learning powers our experiences.

Starting with search engines like Google, deep learning is at the heart of predicting your next word as you type in the search bar. This not only enhances user experience but also refines search accuracy by understanding the context from the previous text. Moving to machine translation tools like Google Translate now allow for real-time translation across multiple languages, making global communication seamless. You can even snap a photo of a text in one language and have it instantly translated to another. Social media platforms leverage deep learning extensively to analyse sentiments expressed in posts across different languages. This analysis helps tailor targeted advertising based on user interactions and preferences. Another fascinating application is chatbots, which are becoming increasingly sophisticated. These bots engage in human-like conversations, providing users with needed information, and are prevalent on websites ranging from telecommunications to government services.

In the professional, realm deep learning helps in document classification and summarisation, for instance.

When hiring managers face piles of resumes, NLP models can quickly filter through them identifying the most relevant candidates based on semantic content analysis. Voice assistance like Apple Siri, Google Assistance, and Amazon's Alexa are perhaps the most relatable applications of deep learning in NLP. They process spoken language converting it to text and executing commands or answering queries in real-time. Furthermore, tools like Grammarly utilise deep learning to enhance writing by correcting grammar and style, ensuring your communication is both clear and correct. Lastly, email applications such as Gmail utilise deep learning for efficient spam filtering and for crafting suggestive responses to emails, which saves time and enhance productivity.

These examples underscore the broad and powerful applications of deep learning in NLP, which significantly improve interaction quality, and information accessibility across various digital platforms.

Video 8: Deep Learning for NLP, What is RNN and How it Works?

Deep learning for NLP. Let's try to understand what is RNN and how it works. In this video, we will discuss natural language processing, a fascinating branch of AI that harness computational algorithms to interpret, analyse, and represent human spoken languages.

NLP allows computers to process and understand human languages in a way that is both meaningful and useful. NLP Technologies power numerous applications we interact with daily. For instance, search engines like Google, use NLP to understand and anticipate search queries, enhancing the relevance of search results. Similarly, voice assistance such as Amazon's Alexa and Apple Siri rely on NLP to interpret spoken commands and provide accurate responses. Moreover, NLP is crucial in enabling automatic text generation, which can be seen in real-time captioning systems that convert spoken language into a written text. These applications showcase how deep learning models are trained to handle complex language-based tasks, including machine translation and dialogue generation. Further bridging the gap between human communication and machine understanding. By equipping machines with the ability to understand and generate human language. NLP is setting new benchmark in how we interact with technology, making it an indispensable tool in the digital era.

In this segment, we will explore recurrent neural networks, or simply called as RNNs, which are pivotal framework in deep learning for natural language processing. RNNs are also similar to CNNs for computer vision but tailored for sequential data processing such as language or time series analysis. RNNs are renowned for their capability in document classification, text generation and many more. Their unique feature is the ability to process sequences. If given a series of words, an RNN can predict the subsequent word. This predictive ability stems from networks design, where the output from each step is fed back into the network as part of the input for the next step. The diagram here illustrates an unrolled, RNN. Each node labelled A represents a batch of neurons, processing input at different timesteps. The input at each time step, which is x_t , is processed to produce an output which is h_t , which then becomes part of the input for the next time step. This sequential processing allows the network to maintain a memory over time, leveraging previous computations, h_{t-1} , h_{t-2} , and so on as inputs along the current input x_t . By effectively capturing information from the entire sequence up to that point.

As you can see, the output from each neuron that is h_t is recycled back into the network, providing a memory effect that allows the network to make informed predictions based on previous data. Unlike Feedforward networks where data flows in one direction. The RNN feedback loop enables it to remember previous inputs. This memory is crucial when the context or sequence of events influence the outcome.

Visualise the RNN as a multiple copies of the same network, each passing information to the next. This structure allows the network to handle variable length input sequence essential for tasks like language translation or speech recognition where the context from earlier in the sequence affects the understanding of predictions with later on.

Recurrent neural networks excel in handling tasks where the prediction depend on short-term context within the data. The figure illustrates an unrolled RNN, showing how it process sequential data over time. Each node labelled A represents the network at a specific timestep processing one input from the sequence and passing its output to the next step. This process begins at time t_0 with the initial x_0 and the corresponding hidden state h_0 , which is then passed as an input to the next node.

This pattern continues with each node receiving an input X and the hidden state from the previous node h until time t_n where N is the final time step in the sequence. For instance, in the text example, the colour of the sky is the RNN receives each word one at a time. 'The' at time t_0 , colour at t_1 , 'of' at t_2 , "the" at t_3 , 'sky' at t_4 and finally is at t_5 . The network must remember relevant information from all previous words to accurately predict the word blue as the next word. Each word influences this hidden state, which carries forward this memory to the sequence context enabling the prediction of next word based on short term dependencies. This process underscores the RNN capability to manage the sequence where the relationship between the item is closely linked over short spans. However, RNN struggles with long-term dependencies, which is a limitation crucial when the context needs for a correct prediction that spans a large gap in the input sequence.

In the example, I have spent 20 years teaching computer science for the underprivileged kids in France. Then we have 20 to 40 words, and then I have the sentence, I can speak fluent, and you need to predict the next word. The answer, which is "French" heavily relies on the word France mentioned much earlier in the text. Traditional RNNs tend to forget such distant information, failing to recall "France" when predicting the language spoken. This limitation stems from the vanishing gradient problem where the influence of information decreases as it travels through each step of the network, making it hard for standard RNNs to maintain long-term dependencies. As a result, while RNNs are powerful for many sequential tasks, their application is limited by the length of dependencies they can effectively remember and utilise for predictions.

To overcome these limitations of RNN people came up with a new deep learning architecture called LSTM Architecture or long short-term memory network. The core purpose of LSTM is to extend the memory of data sequence significantly longer than RNNs can handle, allowing for more extended dependency recognition in sequence data. LSTM achieves this through specialised memory blocks connected into layers each consists of three distinct gates. Input gate, forget gate and output gate each playing a vital role in memory management. The forget gate is crucial as it filters out the unnecessary information from previous step, ensuring only relevant data persists. The input gate decides which of the current input data should be stored in the LSTMs memory state, ensuring that significant new information is retained for future steps. The output gate, which determines what the next output should be based on both the new input and the information already stored in the memory cell. These gates collectively enable the LSTM to retain information over a long period and handle long-term dependencies more effectively than traditional RNNs. This makes LSTM particularly suited for complex tasks like language translation, speech recognition and time series prediction where understanding extended context is important.

We are not going in detail on how these LSTM architecture works or how the weights of LSTM layers are learned because it's beyond the scope of this module.

Instead, we are just giving you a very high-level introduction on the LSTM layer. In this last segment of this video, we discuss how text is transformed into numerical data that are to be used for deep learning applications. Direct textual input is not feasible for deep learning models. Instead, we must convert text into a numeric format. We start with one hot encoding. In this approach, each word in our vocabulary is represented as a vector. Consider our sample vocabulary of nine words. For the word man, we represented with a vector where the position corresponding to man is marked as one, and all other positions are zero. This means if you see a vector with a one at the first position and the remaining zeros, it corresponds to the word man. Each word is thus represented as a unique vector within a high dimensional space. Though this method results in spars and large vectors due to its simplicity.

Next, we explore word embeddings, a more sophisticated method of text representation. Unlike one hot encoding word, embeddings are dense, lower -dimensional, and learn from data.

In our vocabulary, we might define categories like femininity or youth and associate words with these categories to meaningful vectors. For instance, femininity might closely relate to woman and princess embedding these relationships within the vector space.

The difference between one-hot encoding and word embedding is clear. One-hot vectors are simplistic and straightforward leading to high dimensional data. On the other hand, word embeddings are compact, dense, and derived through learning semantic relationships within the text.

Video 9: Keras Example for Text Classification

Keras example for text classification. In this video, we will see an example of how to implement long short-term memory that is LSTM model using TensorFlow and Keras, specifically aimed at sentiment analysis with the IMDB movie review dataset. This tutorial is structured to give you a comprehensive step-by-step walkthrough from data loading and pre-processing to model building, training and evaluation.

Step one, imports and setups. First, we import tensorflow along with necessary components from Keras. Our step involves defining several parameters critical for our models, architecture and training process. First one is the max feature. This specifies the maximum number of words to include in our vocabulary, filtered based on frequency, maxlen, the maximum length of the sequences, which will be our outputs, output size, batch size, number of samples per gradient update during the training. Embedding dimension, the size of the embedding vectors. epochs, which is the number of times the training process will iterate over the entire dataset.

Step two is loading data. We load the IMDB dataset using imdb.load data function, limiting our vocabulary to max features most common words for efficiency and manageability. The dataset is then divided into training and testing set. We further slice these datasets to work computation, as this is a educational demo. Step three, pre-processing the data to ensure that all input sequences in both training and test sets have the same length we employ padding sequences. This is important as LSTM layers require input tensors to be of same dimensions. This function pads or truncate sequences to a fixed length of maxlen.

Step four, model creation and training. Our model is sequential with layers stacked linearly. It starts with an embedding layer, which converts positive integer that is indexes into dense vectors of fixed size. The max features specify the size of the vocabulary. Embedding dimensions is the dimensions of the dense embedding and input length is the length of the input sequences.

The LSTM layer follows with 128 units to capture dependencies in input data. Finally, a deep layer with a sigmoid activation function. Makes the binary prediction whether the review is positive review or negative review. We compile the model and run the model training.

Step five, evaluation. After training, we evaluate the model's performance on the test dataset. This provides us with a loss value and accuracy, giving insight into how well our model performs on unseen data. This example provides a foundational understanding of working with LSTM networks in Keras for NLP task. This also highlights key steps from data handling to model evaluation. Such knowledge is essential for anyone looking to dwell into practical applications of deep learning in natural language processing.

Video 10: Module Summary

This week we embarked on an enlightening journey through the realm of deep learning, exploring its necessity distinction from traditional machine learning and its growing popularity.

We began addressing the fundamental question, why deep learning, unlike conventional machine learning techniques that often require meticulous feature engineering and are limited in handling vast amount of unstructured data. Deep learning shines by leveraging large scale data to automatically discover the representations needed for feature detection or classification. The popularity of deep learning in recent years can largely be attributed to advances in computational power, the availability of massive datasets and significant improvements in training algorithms enabling the training of deep neural networks that are capable of complex representation, learning.

Transitioning to practical applications, our focus shifted to deep learning in computer vision. Here we studied how CNNs play a pivotal role. We also discuss their architecture layers and how they intuitively process and interpret image data. To concretise this, we examined a hands-on Keras example that detailed the steps involved in building and training a CNN for object classification using the CIFAR 10 dataset. This section illustrated not only the setup and training phase but also highlighted key functionalities like convolutional layers and pooling layers, which are instrumental in feature extraction and dimensionality reduction, respectively. Furthermore, our exploration extended to deep learning for natural language processing where we explored its application ranging from text classification to sentiment analysis. The discussion started with RNN and how it's limitation can be overcome by the LSTMs. These models are uniquely designed to handle sequence data by capturing temporal dependencies and avoiding issues like vanishing gradient problem commonly in traditional RNNs.

Through a Keras tutorial, we demonstrated how to implement a simple LSTM for sentiment analysis on the IMDB review dataset. We also showcase how these networks efficiently process and predict based on text data. Thank you.

THE END